

PROJECT REPORT

Smart Student Management & Academic Performance System

A console-based application built in core Java

Course	CSE2006 – Programming in Java
Course Type	LP (3 Credits)
Submitted for	VITyarthi – Build Your Own Project
Student Name	KISLAY KUMAR_____
Registration No.	25BAI10313_____
Programme / Branch	BTECH (AIML)_____
Faculty	DR NILESH KUNHARE_____
GitHub Repository	https://github.com/kislaykumar07/java-project-Smart-Student-Management---Academic-Performance-System
Date of Submission	18 SEPTEMBER 2026_____

I declare that the work presented in this report and in the accompanying repository was carried out by me, and that all external sources used are listed in the References section.

Signature: KISLAY KUMAR_____

Table of Contents

1.	Introduction	3
2.	Problem Statement	3
3.	Objectives	4
4.	Functional Requirements	4
5.	Non-functional Requirements	6
6.	System Architecture	7
7.	Design Diagrams	9
	7.1 Use Case Diagram	9
	7.2 Workflow Diagram	10
	7.3 Sequence Diagrams	11
	7.4 Class Diagrams	12
	7.5 ER / Storage Design	14
8.	Design Decisions and Rationale	15
9.	Implementation Details	17
10.	Screenshots and Results	21
11.	Testing Approach	24
12.	Challenges Faced	26
13.	Learnings and Key Takeaways	27
14.	Future Enhancements	28
15.	References	29

Note: page numbers are approximate and may shift by a page after you insert your own screenshots.

1. Introduction

This report describes a console application I built for CSE2006, Programming in Java. The application is called the Smart Student Management and Academic Performance System. It keeps track of students, the courses they are registered for, and the marks they score, and it turns those marks into grades, GPA, rank lists and subject-wise statistics.

I picked this topic because it is a problem I have actually seen. Every class has a CR or a mentor keeping a spreadsheet of marks, and by the end of the semester that sheet is usually a mess. Names are spelled two different ways, someone has typed 105 out of 100 without noticing, and working out who the top five students are means sorting by hand. None of that is hard to solve in software, but it does need the data to be validated when it goes in, not after.

The whole application is written in core Java with no external libraries and no database server. That was a deliberate choice. It means anyone with a JDK installed can clone the repository and run it with one command, and it also means every feature in the project maps to something from the syllabus rather than to a framework doing the work. Data is stored in CSV files which the program reads and writes through character streams.

The program is organised into three functional modules sitting on a shared core of model classes, a storage class and a few utilities. Module one manages student records. Module two manages the course catalogue and the enrollment links between students and courses. Module three takes the marks and produces the analytics. A background thread saves everything to disk every thirty seconds so that work is not lost if the program is closed abruptly.

2. Problem Statement

Small departments, coaching centres and individual class advisors still keep student records in spreadsheets or paper registers. That approach breaks down in four ways.

The first problem is inconsistent data. Because nothing checks what is typed, the same student can appear as "Aarav Sharma" in one sheet and "aarav s" in another, with a different age in each. Once the records disagree there is no reliable way to decide which one is correct.

The second problem is the absence of validation. A mark of 150 out of 100, a blank name, or an email address with no @ sign will all be accepted silently and discovered much later, usually when results are being finalised and there is no time to fix them.

The third problem is that analysis has to be done by hand. Computing an average is easy enough, but a credit-weighted GPA, a pass percentage per subject and a rank list across the whole class are tedious enough that most people only do them once, at the end of the term.

The fourth problem follows from the third. If nobody looks at the numbers until the end of the semester, a student who is failing one subject in the first internal is not identified until it is too late to help them.

The system described in this report addresses all four. Every input passes through a validator before it is accepted. Marks can only be recorded against an enrollment that actually exists, so a student cannot be given marks in a course they never took. Grades, GPA, rank lists and course statistics are computed on

demand from the stored data, so they are always current and never have to be recalculated by hand.

3. Objectives

The objectives I set for the project at the start were these:

1. To design and build an original console application in core Java that applies the object-oriented concepts covered in the course, rather than demonstrating them one at a time in isolated programs.
2. To implement at least three distinct functional modules with a clear input and output structure and a logical workflow between them.
3. To store and retrieve data reliably across runs using Java I/O streams, without depending on a database server.
4. To validate every piece of user input and to report failures through a custom exception hierarchy instead of crashing or silently accepting bad data.
5. To use multithreading for a genuine purpose — in this case background persistence — rather than as a demonstration that serves no function.
6. To derive meaningful academic analytics from stored marks, including credit-weighted GPA, rank lists and subject-level statistics.
7. To keep the code modular enough that the storage mechanism could later be replaced with JDBC without touching the business logic.
8. To verify the behaviour with an automated test suite and to maintain the work under version control with a readable commit history.

4. Functional Requirements

The application provides three major functional modules. Each one is reachable from the main menu and each has its own sub-menu.

4.1 Module 1 — Student Management

ID	Requirement	Input	Output
FR-1.1	Add a new student record	ID, name, age, email, department, semester	Confirmation, or a validation error
FR-1.2	List all students, sorted by name	Menu selection	Formatted table with a total count
FR-1.3	Search students by partial name, or by name and department	Name fragment (and optionally a department)	Matching records, or a not-found message
FR-1.4	Update any subset of a student's fields	Student ID, then new values (blank keeps the old one)	The updated record
FR-1.5	Delete a student and their grade records	Student ID and a y/n confirmation	Confirmation of cascade deletion

4.2 Module 2 — Course and Enrollment Management

ID	Requirement	Input	Output
FR-2.1	Add a course to the catalogue	Code, title, credits, faculty	Confirmation or error
FR-2.2	List all courses	Menu selection	Catalogue with a total count
FR-2.3	Enroll a student in a course	Student ID, course code	Confirmation, or a duplicate-enrollment error
FR-2.4	Remove an enrollment	Student ID, course code	Confirmation or error
FR-2.5	List the students registered for a course	Course code	List of enrolled students
FR-2.6	Record or update marks for an enrollment	Student ID, course code, marks out of 100	The stored record with its letter grade and pass status
FR-2.7	Delete a course and clear its enrollments	Course code	Confirmation of cascade cleanup

4.3 Module 3 — Academic Performance Analytics

ID	Requirement	Input	Output
FR-3.1	Generate an individual report card	Student ID	Per-course marks, grade, pass/fail, average and GPA
FR-3.2	Produce a rank list ordered by GPA	Menu selection	Ranked table of all students
FR-3.3	Show the top N performers	A count	The first N rows of the rank list
FR-3.4	Compute statistics for one course	Course code	Average, highest, lowest, pass percentage, grade histogram

ID	Requirement	Input	Output
FR-3.5	Display a students-by-courses marks matrix	Menu selection	Grid of marks with a row average
FR-3.6	Display an institute-wide summary	Menu selection	Counts, generation timestamp and the full rank list
FR-3.7	Export the summary and matrix to a file	Menu selection	A timestamped .txt file in the data folder

4.4 Input and output structure

All input is read from the keyboard through a `BufferedReader` wrapped around `System.in`, one prompt at a time. All output is printed to the console as formatted text tables built with `String.format`. The only persistent output is the CSV data files and the exported report, both written through character streams. There is no graphical interface, which keeps the focus on the logic rather than on UI code.

4.5 Workflow

The user starts the program, which loads any saved data from disk and seeds a small sample dataset if none is found. The main menu then appears. From there the user picks one of the three modules, works through its sub-menu, and is returned to the main menu after every operation, whether it succeeded or raised an error. Choosing exit flushes everything to disk and stops the background thread. The order that matters in practice is: add a course, add a student, enroll the student in the course, record marks, and only then are the analytics meaningful. The system enforces that order by refusing to record marks for a student who is not enrolled.

5. Non-functional Requirements

Six non-functional requirements were specified, against the four required by the guidelines. Each one is listed here with the way it is actually met in the code, since a requirement that is stated but not implemented is not worth much.

#	Requirement	Target	How it is met in the implementation
NFR-1	Reliability	No data loss on abnormal termination	A daemon thread flushes all three collections to CSV every 30 seconds. A JVM shutdown hook stops that thread cleanly, and the exit path saves once more before the program ends. Corrupt rows in a CSV file are skipped with a warning rather than aborting the load.
NFR-2	Usability	Usable without a manual	Numbered menus throughout, prompts that state the expected format (for example "Student ID (e.g. S101)"), blank input to keep an existing value while editing, and aligned output tables that stay readable in a standard 80-column terminal.
NFR-3	Maintainability	Storage replaceable without touching business logic	Seven packages with one responsibility each. Every service talks to disk only through FileStorage, so swapping CSV for JDBC means rewriting one class. No service knows how any other service stores its data.
NFR-4	Error handling	No unhandled exception ever reaches the user	A custom checked exception hierarchy rooted at SMSEException. Validation, duplicate and not-found errors are separate subclasses. The console layer catches SMSEException in a single place and prints the message, then returns to the menu.
NFR-5	Performance	Instant response for a class-sized dataset	Students, courses and grade records are held in LinkedHashMap, so lookup by key is $O(1)$. Report generation is linear in the number of records. For a few hundred students every operation completes well under a second.
NFR-6	Resource efficiency	Runs on a plain JDK with no downloads	Zero external dependencies, including for testing. Streams are buffered and closed in finally blocks. The auto-save thread sleeps between saves rather than polling, and is a daemon so it never keeps the JVM alive.

6. System Architecture

The application follows a layered architecture. Each layer only talks to the one below it, which is what makes the storage mechanism replaceable and what makes the services testable without a console attached.

6.1 The layers

Presentation layer. Main wires the objects together, loads saved data and starts the background thread. ConsoleUI owns every prompt, every menu and every printed message. No other class in the project calls System.out.println for user-facing output, and no service class reads from the keyboard. Keeping that boundary strict is what allowed the test suite to drive the services directly.

Service layer. Five classes hold the business rules. StudentService, CourseService and GradeService each own one collection and the operations on it. ReportService owns no data of its own; it reads from the other three and formats the results. AutoSaveService is a thread that periodically asks the first three to write themselves to disk.

Domain model. Six small classes that hold data and the rules belonging to that data. A GradeRecord knows how to turn its own marks into a percentage and a letter grade; nothing outside the class needs to know the grade boundaries.

Persistence layer. One class, FileStorage, which reads and writes lists of text lines to a named file in a data directory. It knows nothing about students or marks, only about lines of text.

Cross-cutting. InputValidator and MathUtil are stateless utility classes with private constructors. The exception package defines the four error types used throughout.

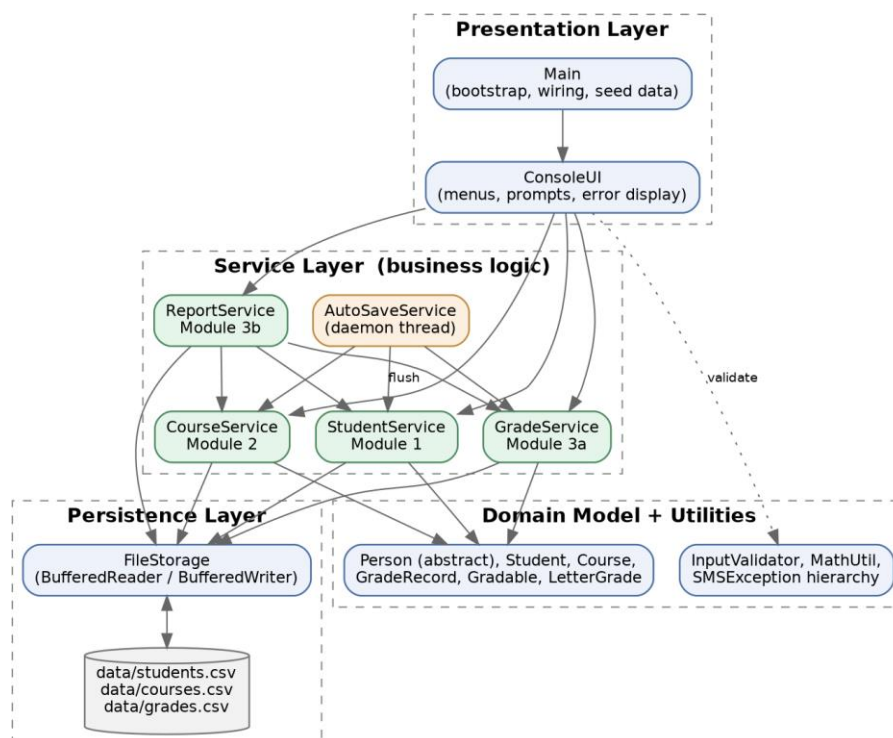


Figure 1: System architecture. Arrows show the direction of dependency; the dotted arrow from ConsoleUI marks validation calls.

6.2 Why the dependencies point this way

CourseService depends on StudentService rather than the other way round, because enrolling a student is an operation on a course roster that needs to confirm the student exists. GradeService depends on both, because a mark is only valid if there is a student, a course, and an enrollment linking them. ReportService sits on top of all three and depends on nothing below the service layer except MathUtil. Nothing points back upwards, so there are no cycles, and each class can be constructed in a test with just the objects it needs.

6.3 Package structure

```
src/com/sms/  
  app/      Main.java, ConsoleUI.java  
  model/    Person.java, Student.java, Course.java,  
            GradeRecord.java, Gradable.java, LetterGrade.java  
  service/  StudentService.java, CourseService.java, GradeService.java,  
            ReportService.java, AutoSaveService.java  
  storage/  FileStorage.java  
  exception/ SMSEException.java, InvalidInputException.java,  
            DuplicateRecordException.java, RecordNotFoundException.java  
  util/     InputValidator.java, MathUtil.java  
  test/     SimpleTests.java
```

That is seventeen classes across seven user-defined packages, against the five to ten modules the project guidelines ask for.

7. Design Diagrams

7.1 Use Case Diagram

Three actors use the system. The administrator maintains the student master list and looks at institute-level reports. Faculty maintain the course catalogue, enroll students and enter marks. The class mentor mainly reads report cards to find students who need help. In the current console version there is no login, so these are roles rather than enforced permissions; adding real authentication is listed under future enhancements.

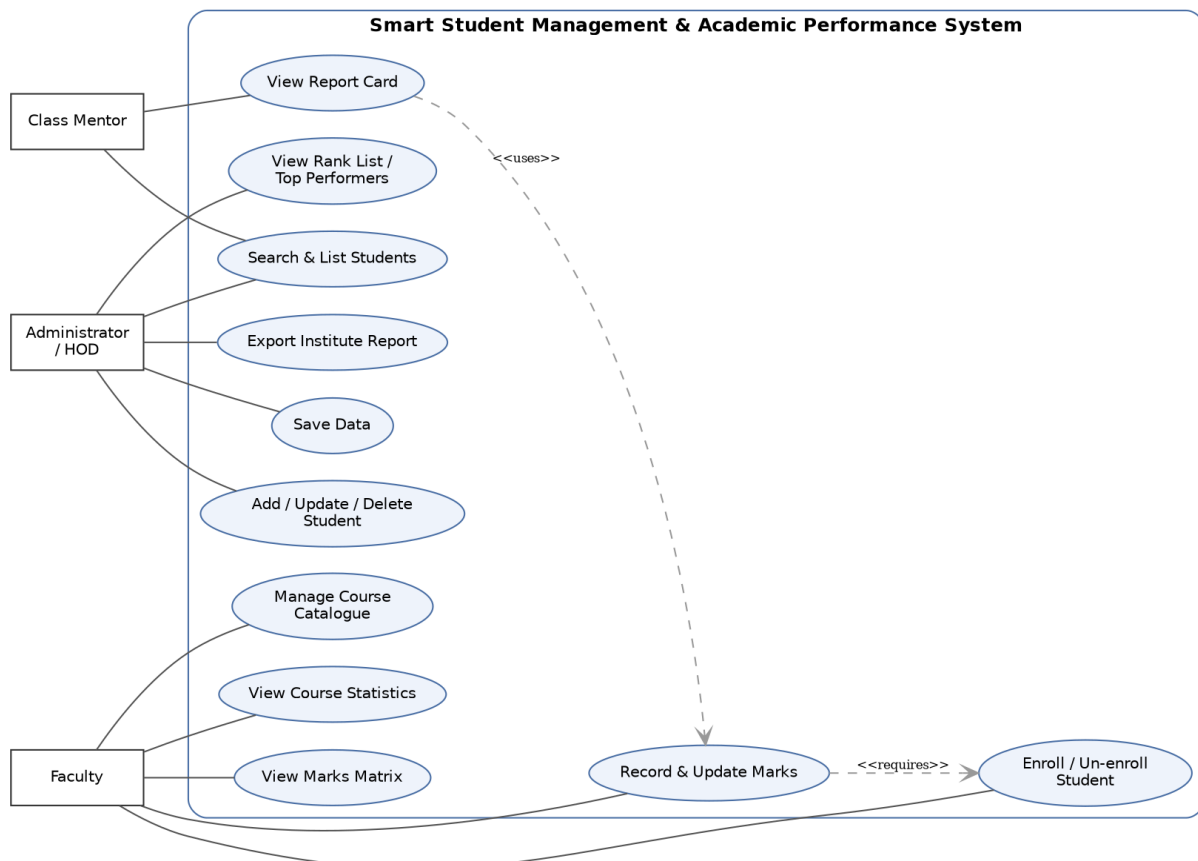


Figure 2: Use case diagram showing the three actors and eleven use cases.

7.2 Process and Workflow Diagram

This diagram traces one full run of the program, from startup through the menu loop to shutdown. The point worth noticing is that both the success path and the error path return to the same menu, so a mistake never ends the session.

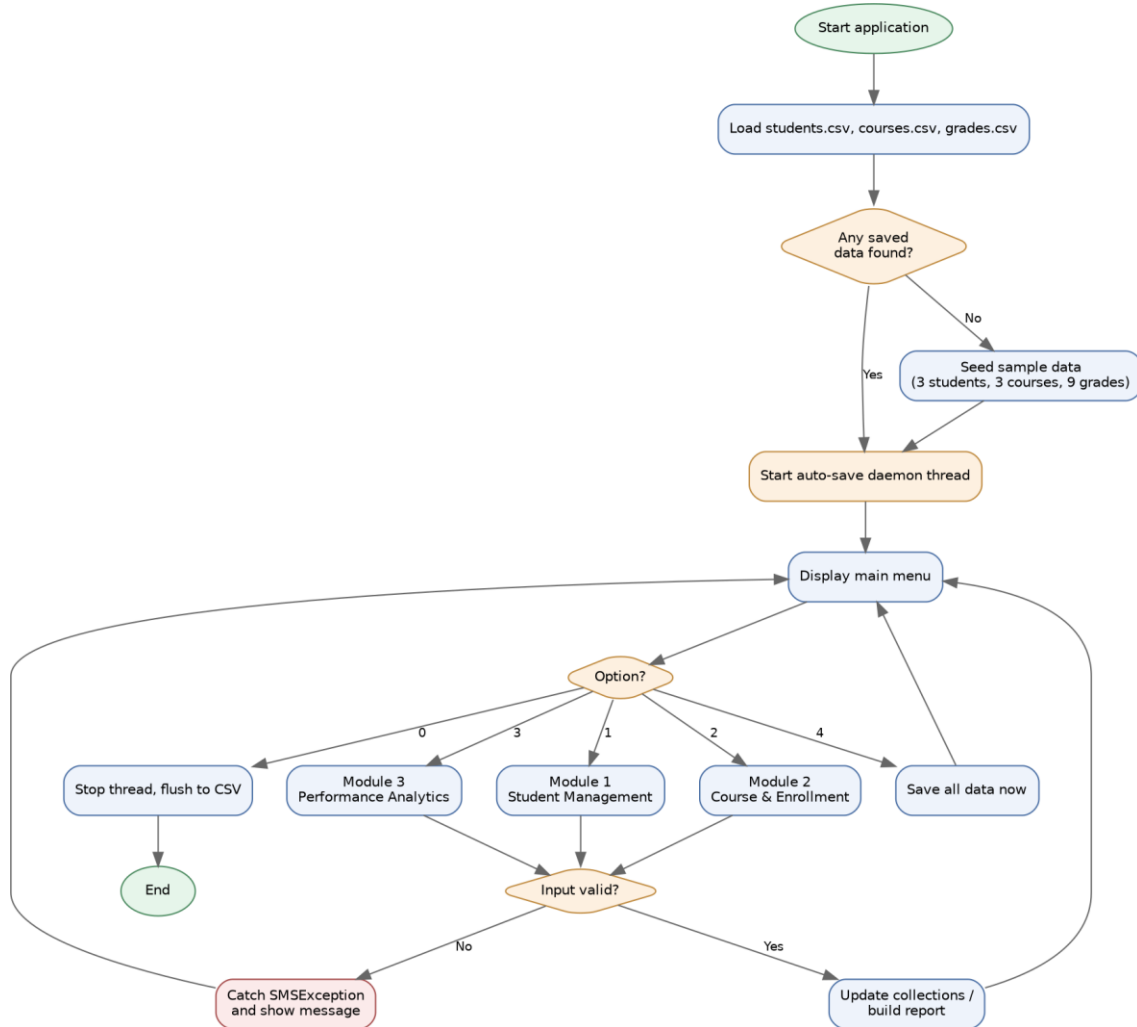


Figure 3: Process flow for a complete session.

7.3 Sequence Diagrams

The first sequence diagram covers recording marks, which is the operation with the most validation behind it. The input is validated for format before the service is called at all, then the service independently confirms that the student exists, that the course exists, and that an enrollment links them. Only then is the record stored. The dashed red arrow at the bottom is the auto-save thread, which runs on its own schedule and is not part of the user's request.

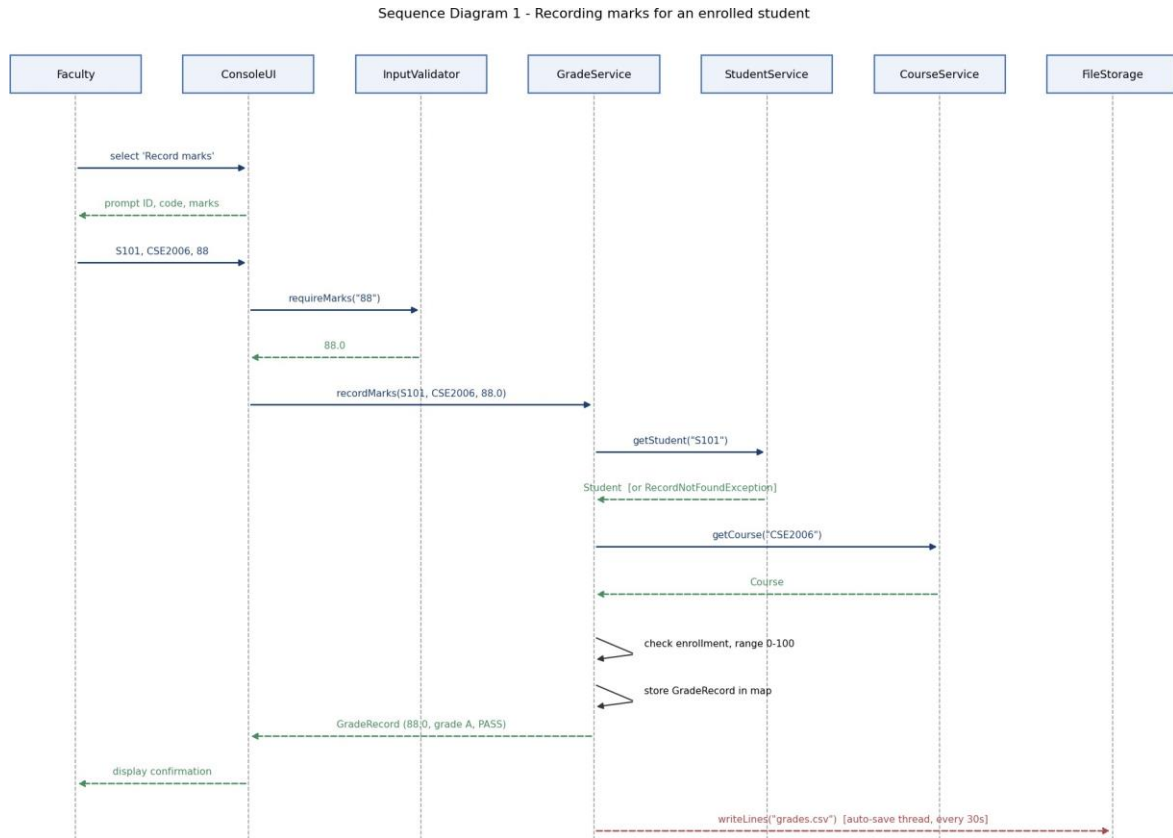


Figure 4: Sequence diagram for recording marks.

The second diagram covers report card generation, which is a read-only operation that gathers data from three services and formats it.

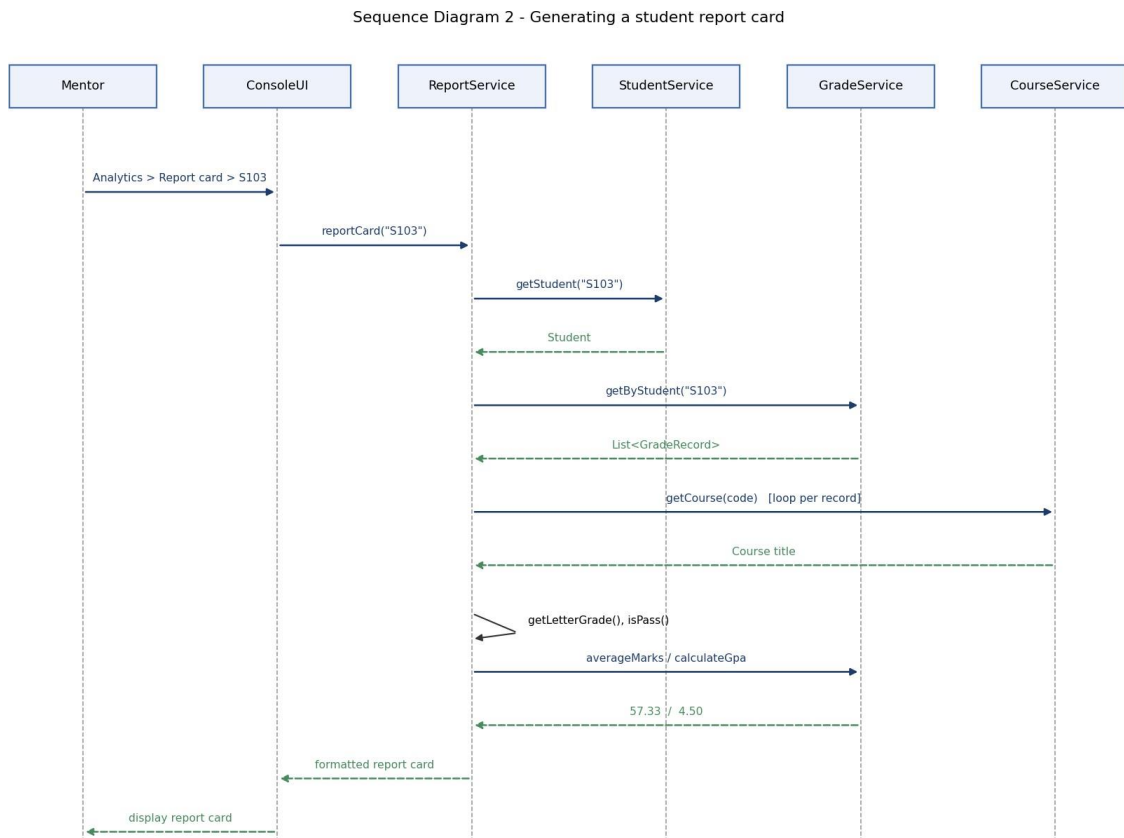


Figure 5: Sequence diagram for generating a report card.

7.4 Class Diagrams

The model classes are shown first. Student extends the abstract Person class, GradeRecord implements the Gradable interface, and LetterGrade is an enum that both the record and the reports depend on.

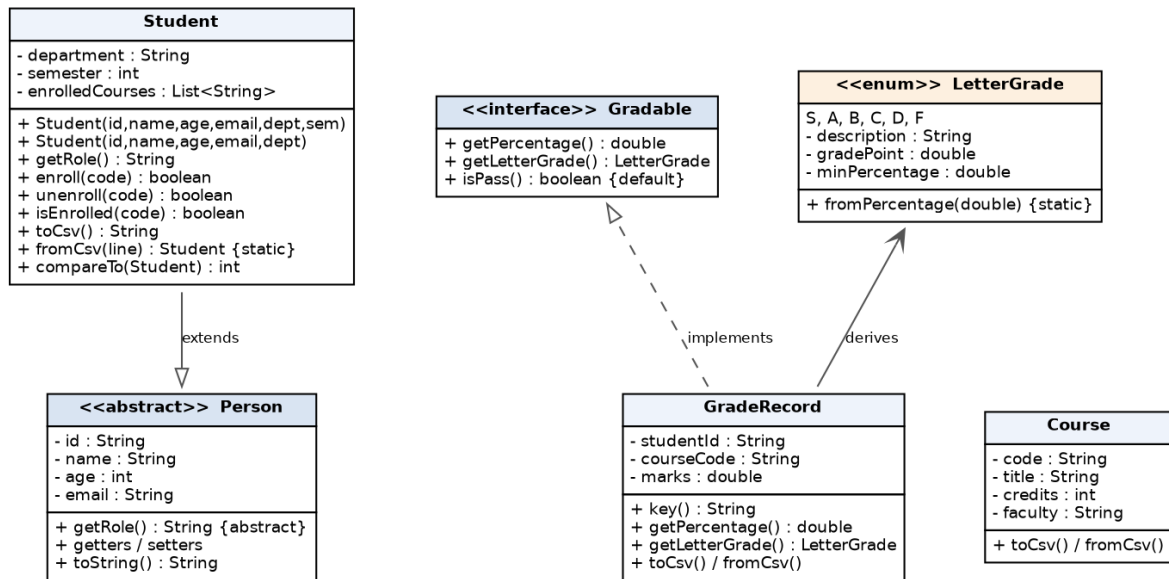


Figure 6: Class diagram of the domain model.

The service and infrastructure classes are shown separately to keep the diagram readable.

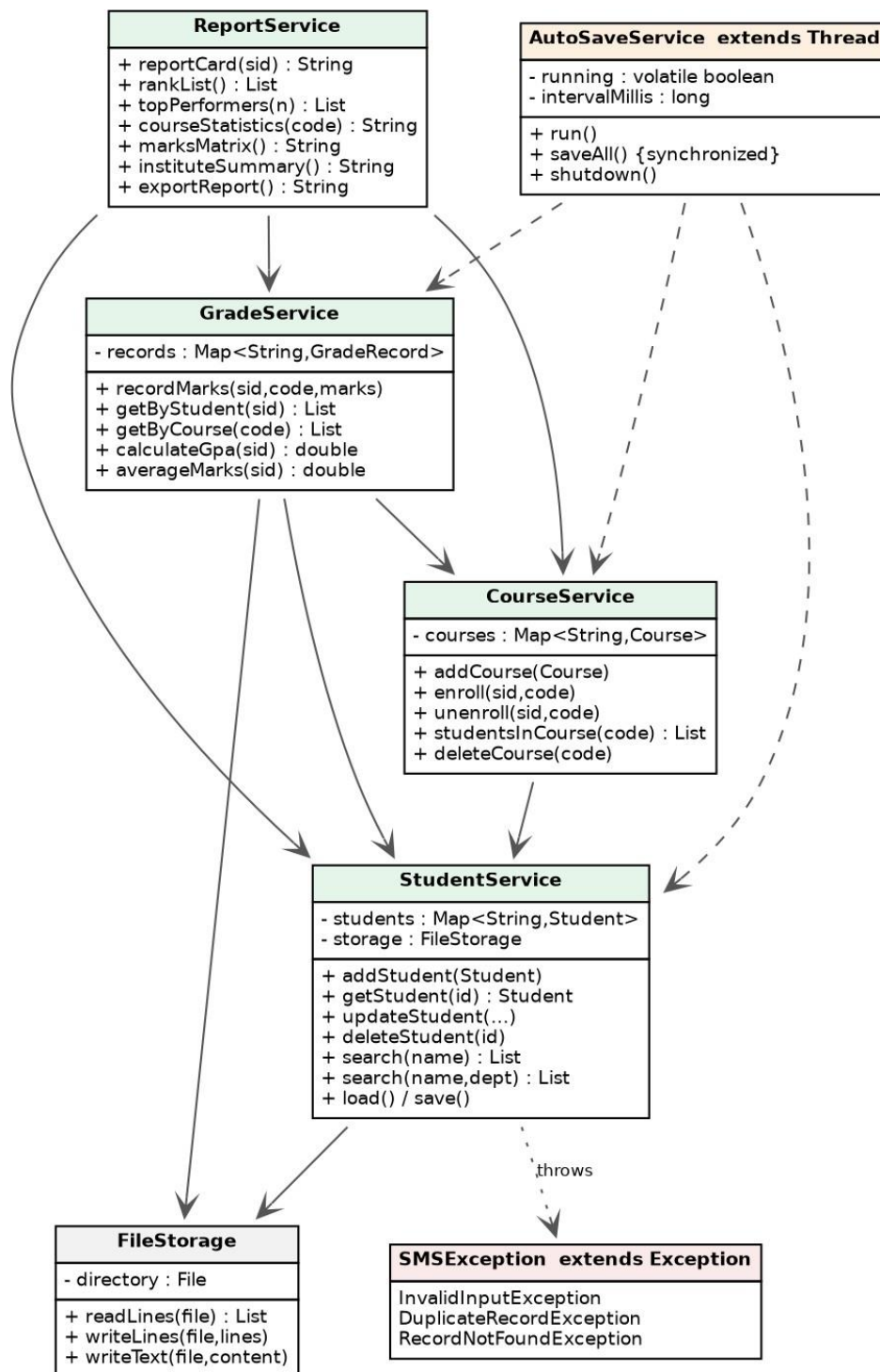


Figure 7: Class diagram of the service, storage and exception layers.

7.5 ER and Storage Design

There are three entities and one relationship. A student registers for many courses and a course is taken by many students, so the enrollment between them is a many-to-many relationship. Each enrollment may or may not have a grade record attached, because a student can be enrolled before any marks have been entered.

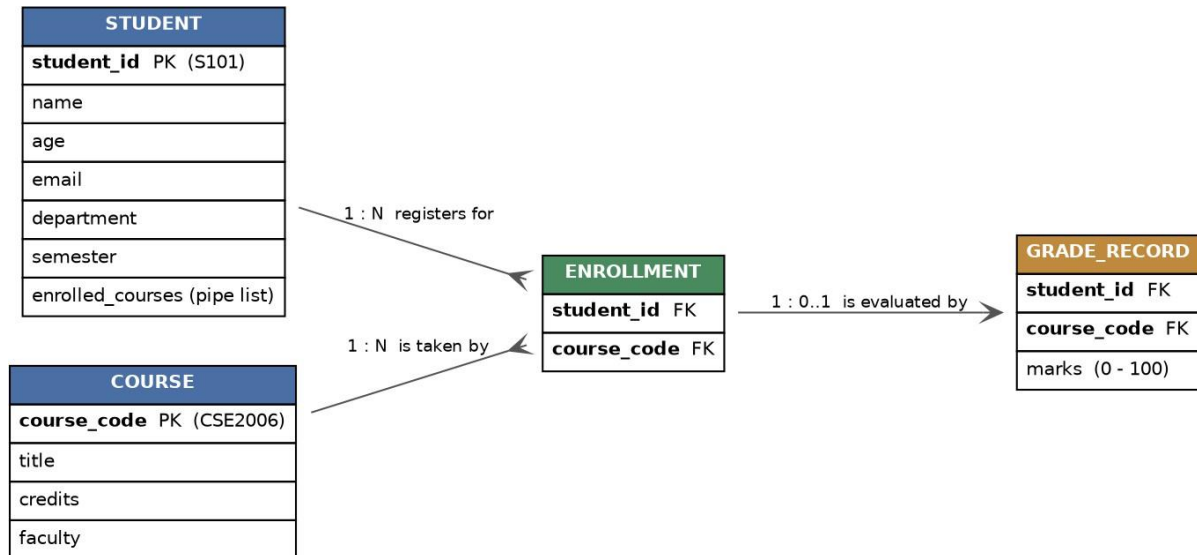


Figure 8: Entity relationship diagram.

Because the storage is flat files rather than a relational database, the enrollment relationship is denormalised into a pipe-separated column inside the student row. This is a compromise: it removes the need for a join, but it means the enrollment list and the grade file have to be kept consistent in application code, which is handled by the cascade logic in `deleteStudent` and `deleteCourse`.

File	Format	Example row
students.csv	id, name, age, email, department, semester, course1 course2	S101,Aarav Sharma,19,aarav@vit.ac.in,CSE,3,CSE2006 CSE2001
courses.csv	code, title, credits, faculty	CSE2006,Programming in Java,3,Dr. Ashwin
grades.csv	studentId, courseCode, marks	S101,CSE2006,88.0
report_*.txt	Plain text	Generated on demand by the export option

In memory the grade records are keyed by the string `studentId + "::" + courseCode`, which reproduces the composite primary key of the `GRADE_RECORD` entity and guarantees one record per student per course.

8. Design Decisions and Rationale

These are the choices that shaped the project, and the reasoning behind each. Several of them were changed partway through, and I have said so where that happened.

8.1 Flat files instead of JDBC

Unit 5 of the syllabus covers JDBC and JPA, so using a database would have been the obvious route. I chose CSV files instead, for two reasons. First, a project that needs MySQL installed and a schema created before it will run is a project that is difficult to evaluate and difficult to share. Second, Unit 4 covers I/O streams and Reader/Writer, and flat files exercise that material properly. To avoid losing the JDBC path entirely, all file access goes through a single `FileStorage` class, so migrating to a database means writing one new class with the same three methods and changing one line in `Main`. The classes that hold the business rules would not change at all.

8.2 An abstract `Person` class with only one subclass

Right now `Student` is the only class that extends `Person`, and it would be fair to ask why the abstraction exists at all. The reason is that `Faculty` is the obvious next entity, and it shares identity, name, age and email exactly. Putting those fields in a base class now means adding `Faculty` later touches no existing code. `Person` is abstract because a bare `Person` has no meaning in this system, and `getRole` is abstract so that every concrete subclass has to declare what it is.

8.3 A checked exception hierarchy rather than return codes

An early version of `StudentService` returned boolean from `add` and `delete`, and null from lookups. That turned out badly. The console layer could tell that something failed but not why, so the error messages were vague, and a forgotten null check caused a `NullPointerException` during testing. I replaced it with three checked exceptions under a common `SMSEException` parent. The service now throws an exception carrying a specific message, and because they all share a parent, the console layer catches `SMSEException` once per menu instead of catching three types everywhere. Making them checked rather than unchecked was deliberate: the compiler then forces the caller to deal with the failure.

8.4 Marks are rejected unless an enrollment exists

The system refuses to store marks for a student who is not enrolled in that course. This adds a step for the user, who has to enroll before entering marks, but it is the rule that keeps the data honest. Without it, a typo in a course code would silently create marks for a course the student never took, and the GPA would be computed from it.

8.5 Credit-weighted GPA rather than a plain average

A simple average of grade points treats a one-credit elective the same as a four-credit core subject. The GPA here multiplies each grade point by the course credits, sums that, and divides by total credits, which is how universities actually compute it. This is also why `GradeService` needs a reference to `CourseService`: it has to look up the credits for each course. If a course is deleted after marks were recorded, the calculation falls back to a default weight of three rather than failing.

8.6 Auto-save on a background thread

Writing all three files after every single change would mean three file writes for adding one student, which is wasteful. Never writing until exit risks losing everything if the terminal is closed. The compromise is a daemon thread that sleeps thirty seconds and then flushes. Because that thread reads the same maps the menu operations write to, every public method on the three data services is synchronized, and the running flag in `AutoSaveService` is volatile so the shutdown signal is visible to the thread immediately. The thread is a daemon so it can never keep the JVM alive after the user has exited.

8.7 Sample data on first run

When the program starts and finds no saved data, it seeds three students, three courses and nine grade records. This was added after I demonstrated an early version and realised that every analytics screen was empty until about twenty inputs had been typed. Seeding means every menu option produces something meaningful on the very first run.

8.8 A test runner written by hand

JUnit would be the normal choice, but adding it means adding jars to the repository or a build tool, and that conflicts with the goal of running on a plain JDK. `SimpleTests` is a class with a main method, a `check(label, condition)` helper, pass and fail counters, and a non-zero exit code if anything fails. It is not a replacement for a real framework, but it runs with one command and it catches regressions, which is what I needed.

9. Implementation Details

9.1 Environment

Item	Detail
Language	Java, source level 8 (developed and tested on JDK 21)
Build	javac from the command line via run.sh / run.bat; no build tool
External libraries	None
Storage	CSV text files in a data/ directory
Version control	Git, hosted on GitHub
Lines of code	Approximately 2,000 across 17 classes

9.2 Mapping of syllabus concepts to code

The table below shows where each unit of the syllabus is actually used. Nothing here was added purely to tick a box; each concept is used because it was the natural solution for that part of the problem.

Unit	Concept	Where it is used
1	Data types, operators, expressions	Mark and GPA arithmetic throughout GradeService and MathUtil
1	Input and output	BufferedReader over System.in, System.out formatting in ConsoleUI
1	if-else, switch, for, while, break	Menu dispatch in ConsoleUI, matrix construction in ReportService
2	Classes, objects, constructors	Every model class; Student has two overloaded constructors
2	Access modifiers, encapsulation	All model fields are private with getters; enrolledCourses is returned as a defensive copy
2	this and final keywords	Constructor chaining with this(...) in Student; final id fields; final utility classes
2	Inheritance and super	Student extends Person and calls super(...) in its constructor
2	Method overriding	getRole, toString, compareTo, and Thread.run in AutoSaveService
2	Method overloading	MathUtil.sum and average for List and array; StudentService.search with one and two arguments
2	Abstract class and interface	Person is abstract; Gradable is an interface with a default isPass method
2	Polymorphism	GradeRecord handled through the Gradable type; Student handled through Person
2	enum with a constructor	LetterGrade holds a description, grade point and minimum percentage for each constant
2	Recursion	MathUtil.sum(List, int) sums marks recursively with an index as the base case
2	Anonymous class	The Comparator in ReportService.rankList and the shutdown hook Thread in Main

Unit	Concept	Where it is used
3	Exceptions and custom exceptions	The four classes in the exception package; try-catch blocks in ConsoleUI and Main
3	throw and throws	Every service method declares the exceptions it can raise
3	Multiple catch blocks	ConsoleUI.start catches SMSException and IOException separately
3	Multithreading and thread life cycle	AutoSaveService extends Thread; new, runnable, timed-waiting and terminated states are all exercised
3	Synchronization	All public service methods and saveAll are synchronized; the running flag is volatile
3	User-defined packages	Seven packages under com.sms
4	Strings and string methods	CSV parsing with split, String.join, String.format, StringBuilder
4	One and two dimensional arrays	The seed marks table in Main; the marks matrix in ReportService
4	Collections framework	LinkedHashMap for all three stores, ArrayList for results, Collections.sort with Comparable and Comparator
4	I/O streams and Reader/Writer	FileStorage uses BufferedReader/FileReader and BufferedWriter/FileWriter
5	JDBC and JPA	Not implemented. Storage is isolated behind FileStorage so that a JDBC implementation can be substituted; see section 14.

9.3 Selected code

The grade enum. Each constant carries its own description, grade point and lower boundary, so the mapping from percentage to grade is data rather than a chain of if statements spread across the code.

```
public enum LetterGrade {
    S("Outstanding", 10.0, 90.0), A("Excellent", 9.0, 80.0),
    B("Good", 8.0, 70.0), C("Average", 7.0, 60.0),
    D("Pass", 6.0, 50.0), F("Fail", 0.0, 0.0);

    private final String description;
    private final double gradePoint;
    private final double minPercentage;

    LetterGrade(String description, double gradePoint, double minPercentage) {
        this.description = description;
        this.gradePoint = gradePoint;
        this.minPercentage = minPercentage;
    }

    public static LetterGrade fromPercentage(double percentage) {
        for (LetterGrade grade : values()) {
            if (percentage >= grade.minPercentage) { return grade; }
        }
        return F;
    }
}
```

Validation before storage. recordMarks performs three independent checks and throws a specific exception for each failure, so the user is told exactly what went wrong.

```
public synchronized GradeRecord recordMarks(String studentId, String courseCode,
                                             double marks)
    throws RecordNotFoundException, InvalidInputException {
    Student student = studentService.getStudent(studentId); // may throw
    Course course = courseService.getCourse(courseCode); // may throw
    if (!student.isEnrolled(course.getCode())) {
        throw new RecordNotFoundException(
            student.getName() + " is not enrolled in " + course.getCode() + ".");
    }
    if (marks < 0 || marks > GradeRecord.MAX_MARKS) {
        throw new InvalidInputException("Marks must be between 0 and 100.");
    }
    GradeRecord record = new GradeRecord(student.getId(), course.getCode(), marks);
    records.put(record.key(), record);
    return record;
}
```

Credit-weighted GPA. Each grade point is weighted by the credits of the course before averaging.

```
public synchronized double calculateGpa(String studentId) {
    double weightedPoints = 0.0;
    int totalCredits = 0;
    for (GradeRecord record : getByStudent(studentId)) {
        int credits;
        try {
            credits = courseService.getCourse(record.getCourseCode()).getCredits();
        } catch (RecordNotFoundException ex) {
            credits = 3; // course deleted later: assume default weight
        }
        weightedPoints += record.getLetterGrade().getGradePoint() * credits;
        totalCredits += credits;
    }
    if (totalCredits == 0) { return 0.0; }
    return MathUtil.round(weightedPoints / totalCredits, 2);
}
```

Recursion. The public sum method hides the index parameter, and the private overload does the recursive work with an empty list as its base case.

```
public static double sum(List<Double> values) {
    return sum(values, 0);
}

private static double sum(List<Double> values, int index) {
    if (values == null || index >= values.size()) {
        return 0.0; // base case
    }
    return values.get(index) + sum(values, index + 1); // recursive case
}
```

The background thread. The run method is the thread body; shutdown sets the volatile flag and interrupts the sleep so that the thread stops promptly instead of finishing its thirty-second wait.

```
@Override
public void run() {
    while (running) {
        try {
            Thread.sleep(intervalMillis); // RUNNABLE -> TIMED_WAITING
            saveAll();
        } catch (InterruptedException ex) {
            Thread.currentThread().interrupt();
            break; // graceful shutdown
        }
    }
}

public synchronized void saveAll() {
    try {
        studentService.save();
        courseService.save();
        gradeService.save();
        saveCount++;
    } catch (IOException ex) {
        System.err.println("Auto-save failed: " + ex.getMessage());
    }
}
```

The two-dimensional marks matrix. A double array of students by courses is filled with -1 to mean "not evaluated", then populated from the grade records and rendered as a table.

```
double[][] matrix = new double[students.size()][courses.size()];
for (int i = 0; i < students.size(); i++) {
    for (int j = 0; j < courses.size(); j++) {
        matrix[i][j] = -1; // -1 marks "not evaluated"
    }
}
for (int i = 0; i < students.size(); i++) {
    for (GradeRecord record : gradeService.getByStudent(students.get(i).getId())) {
        for (int j = 0; j < courses.size(); j++) {
            if (courses.get(j).getCode().equals(record.getCourseCode())) {
                matrix[i][j] = record.getMarks();
            }
        }
    }
}
```

10. Screenshots and Results

The output below was captured from an actual run of the program on the seeded sample dataset. Replace these blocks with screenshots from your own terminal before submitting.

10.1 Startup and main menu

```
Loaded 3 student(s) and 3 course(s) from /home/user/sms/data
=====
SMART STUDENT MANAGEMENT & ACADEMIC
PERFORMANCE SYSTEM (Java Console Application)
=====

===== MAIN MENU =====
1. Student Management
2. Course & Enrollment
3. Performance Analytics
4. Save data now
0. Exit
Choose an option:
```

10.2 Module 1 — listing students

ID	NAME	AGE	DEPARTMENT	SEM	COURSES
S101	Aarav Sharma	19	CSE	Sem-3	Courses:3
S102	Diya Verma	20	CSE	Sem-3	Courses:3
S103	Kabir Nair	19	IT	Sem-3	Courses:3

Total: 3 student(s).

10.3 Module 3 — individual report card

```
-----
REPORT CARD - Kabir Nair (S103)
Department : IT    Semester : 3
-----
```

CODE	COURSE	MARKS	GRADE	RESULT
CSE2006	Programming in Java	45.00	F	FAIL
CSE2001	Data Structures	58.00	D	PASS
MAT2002	Discrete Mathematics	69.00	C	PASS

```
-----
Average marks : 57.33
GPA (10 scale): 4.50
-----
```

This student's GPA is 4.50 rather than the plain average of the grade points (4.67) because Data Structures carries four credits while the other two carry three, and the weaker Java result is weighted accordingly.

10.4 Module 3 — rank list by GPA

RANK	NAME	DEPARTMENT	AVG	GPA
1	Aarav Sharma	CSE	85.33	9.10
2	Diya Verma	CSE	73.33	7.90
3	Kabir Nair	IT	57.33	4.50

10.5 Module 3 — course statistics

 COURSE STATISTICS - CSE2006 : Programming in Java

Students evaluated : 3
 Average marks : 69.00
 Highest marks : 88.00
 Lowest marks : 45.00
 Pass percentage : 66.67%
 Grade distribution :

S
 A # 1
 B # 1
 C
 D
 F # 1

10.6 Module 3 — marks matrix

 MARKS MATRIX

STUDENT	CSE2006	CSE2001	MAT2002	AVG
Aarav Sharma	88.0	92.0	76.0	85.33
Diya Verma	74.0	65.0	81.0	73.33
Kabir Nair	45.0	58.0	69.0	57.33

10.7 Validation and error handling

Entering a badly formatted student ID:

Student ID (e.g. S101): 101
 [ERROR] Student ID must be S followed by 3-5 digits, e.g. S101.

Entering marks outside the valid range:

Marks out of 100: 150
 [ERROR] Marks must be between 0 and 100.

Choosing a menu option that does not exist:

Choose an option: 9
 Invalid option. Please enter 0-4.

In every case the program prints one line and returns to the menu. Nothing crashes and no stack trace is shown to the user.

10.8 Export and persistence

Choose an option: 7
 Report exported to data/report_20260917_160719.txt

 Saving data before exit...
 Goodbye!

After exit the data directory contains students.csv, courses.csv, grades.csv and the exported report. Restarting the program reloads the three CSV files and the previous session's data is intact.

11. Testing Approach

11.1 Strategy

Testing was done at two levels. Automated tests drive the service classes directly and check the results in code. Manual tests exercise the console menus, which the automated suite deliberately does not touch, because ConsoleUI reads from System.in and testing it would mean simulating keyboard input for very little benefit.

The automated suite is the class `com.sms.test.SimpleTests`. It has a main method, a `check(label, condition)` helper that increments a pass or fail counter and prints a line, and it exits with a non-zero status if anything failed, so it can be wired into a build later. There are 38 checks across eight groups.

Group	What it covers	Checks
Validators	ID format, empty text, email format, out-of-range marks	4
MathUtil	Recursive sum, both overloads of average, max, min, rounding	5
Grade mapping	Percentage to letter grade at each boundary, code normalisation, pass and fail flags	7
CSV round trip	Serialising a student and reading it back, including its course list	4
Student CRUD	Add, update, both search overloads, delete	5
Error paths	Duplicate student rejected, missing student reported	2
Enrollment and GPA	Marks blocked before enrollment, record storage, average, credit-weighted GPA, course roster	5
Reporting	Rank ordering, top-N limit, report card, course statistics, matrix, summary	6

11.2 Result

```
Running validation tests...

[PASS] valid student id accepted
[PASS] empty name rejected
[PASS] bad email rejected
[PASS] marks above 100 rejected
[PASS] recursive sum
[PASS] average of list
[PASS] overloaded array average
...
[PASS] course statistics has pass percentage
[PASS] marks matrix rendered
[PASS] institute summary rendered

-----
PASSED: 38   FAILED: 0
-----
```

11.3 The GPA test in detail

The most useful test is the one for credit-weighted GPA, because it checks arithmetic that is easy to get subtly wrong. A student scores 95 in a three-credit course, which is an S worth 10 points, and 75 in a four-credit course, which is a B worth 8 points. The expected GPA is therefore $(10 \times 3 + 8 \times 4) / 7$, which is 8.86. The test computes that expression independently and compares it with the value the service returns.

```
courseService.enroll("S501", "CSE2006");
courseService.enroll("S501", "CSE2001");
gradeService.recordMarks("S501", "CSE2006", 95); // S -> 10 points, 3 credits
gradeService.recordMarks("S501", "CSE2001", 75); // B -> 8 points, 4 credits

check("average marks", gradeService.averageMarks("S501") == 85.0);
double expectedGpa = MathUtil.round((10 * 3 + 8 * 4) / 7.0, 2);
check("credit weighted gpa", gradeService.calculateGpa("S501") == expectedGpa);
```

11.4 Manual test cases

#	Test case	Expected result	Status
1	Add a student with ID "101"	Rejected: ID must be S followed by 3 to 5 digits	Pass
2	Add a student with email "abc"	Rejected: email format is not valid	Pass
3	Add a student whose ID already exists	Rejected: student already exists	Pass
4	Add a student aged 12	Rejected: age must be between 15 and 60	Pass
5	Record marks for a student who is not enrolled	Rejected: not enrolled in that course	Pass
6	Enter marks of 150	Rejected: marks must be between 0 and 100	Pass
7	Enroll the same student in the same course twice	Rejected: already enrolled	Pass
8	Request a report card for an unknown ID	Rejected: no student found with that ID	Pass
9	Update a student leaving every field blank	Record unchanged, no error	Pass
10	Delete a student who has marks recorded	Student and their grade records both removed	Pass
11	Delete a course students are enrolled in	Course removed and enrollments cleared from every student	Pass
12	Add data, exit, then restart the program	Data reloaded from the CSV files	Pass
13	Run analytics with no marks recorded	"No marks recorded yet" instead of a division-by-zero error	Pass
14	Delete the data directory and restart	Directory recreated and the sample dataset seeded	Pass
15	Corrupt one line of students.csv by hand, then restart	Bad row skipped with a warning; the rest load normally	Pass

11.5 What is not tested

ConsoleUI has no automated coverage, so a menu wiring mistake would only be caught manually. Concurrency between the auto-save thread and menu operations is handled with synchronized methods but is not tested under load, since reproducing a race condition reliably is difficult without a proper concurrency testing tool. Both gaps are acknowledged rather than hidden.

12. Challenges Faced

12.1 Deciding where the business rules belong

My first version put the enrollment check inside ConsoleUI, so the menu verified that the student was enrolled before calling GradeService. That worked until I wrote the test suite, which calls the service directly and therefore skipped the check entirely and happily stored invalid marks. The lesson was that validation of business rules has to live in the service, not in the interface, because the interface is only one of the callers. Format validation stayed in the UI, since that genuinely is about the input rather than the rule.

12.2 Commas inside CSV fields

I chose comma-separated storage without thinking about what happens when a field contains a comma. Entering a faculty name as "Dr. Ashwin, M" wrote a row with five fields instead of four, and the next load threw a NumberFormatException while parsing what it thought was the credits column. A full CSV parser with quoting was more than the project needed, so I handled it in two parts: split with a limit and a negative argument so trailing empty fields survive, and wrap each row's parsing in a try-catch so one malformed line is skipped with a warning instead of aborting the entire load. The proper fix, escaping or a different delimiter, is noted in future enhancements.

12.3 The auto-save thread and shared data

The background thread reads the same maps that menu operations write to. In an early run I saw a ConcurrentModificationException while a save happened during a listing operation. Making every public method on the three data services synchronized fixed it. Getting the thread to stop cleanly was a separate problem: setting a boolean flag was not enough, because the thread could be in the middle of a thirty-second sleep and would not notice for half a minute. Marking the flag volatile and calling interrupt() in shutdown() solved it, and taught me the difference between a thread that stops eventually and one that stops promptly.

12.4 Cascading deletes without a database

A relational database handles referential integrity with a foreign key. Here, deleting a student left orphaned rows in grades.csv, and deleting a course left dead course codes inside student records. Both had to be handled explicitly: deleteStudent also calls removeRecordsOfStudent, and deleteCourse loops over every student and un-enrolls them. This is exactly the sort of thing a database would do for free, and it was the moment I understood concretely why referential integrity is a feature people pay for.

12.5 Getting the GPA right

My first implementation averaged the grade points directly. Checking it against a real grade sheet showed the numbers were slightly off for any student whose courses had different credit values. Rewriting it to weight each grade point by credits fixed the calculation but introduced a new dependency from GradeService to CourseService, and then a new edge case: what should happen to a student's GPA if a course they were graded in gets deleted? I chose to fall back to a default weight rather than throw, because refusing to show a report card over a deleted course would be worse for the user than showing a slightly approximate number.

12.6 Formatting console output that stays aligned

Aligning columns with tab characters worked until a student had a long name, at which point the whole table skewed. Switching to `String.format` with explicit field widths fixed the alignment, and long names are now truncated with a trailing dot so a single long entry cannot break the layout.

13. Learnings and Key Takeaways

Abstraction is worth the effort only when it earns its place. Putting every file operation behind FileStorage felt like unnecessary ceremony while I was writing it. It stopped feeling that way when I worked out that switching to JDBC would touch one class. Equally, I now think the Person abstraction is only justified because Faculty is a realistic next step; if it were not, the extra class would be dead weight.

Exceptions carry information that return values do not. Going from boolean returns to a checked exception hierarchy improved the code twice over. The error messages got specific, and the compiler started forcing me to handle failures I would otherwise have forgotten. Having the exceptions share a parent is what made the console layer simple, because one catch block handles every domain error.

Threads are easy to start and hard to stop. Extending Thread and overriding run took a few minutes. Making the thread exit promptly, avoid corrupting shared state, and never prevent the JVM from shutting down took considerably longer, and taught me what volatile, synchronized, interrupt and daemon actually do rather than just what they are called.

Tests find design problems, not just bugs. The enrollment check that lived in the wrong layer was invisible until a test called the service directly. Writing tests forced me to make the services usable without a console, and that constraint improved the structure of the whole project.

Flat files teach you what a database does. Every cascade delete I had to write by hand, and every consistency problem between the enrollment list and the grade file, is something a relational database solves as a matter of course. I understand foreign keys and referential integrity better for having done without them for a few weeks.

Small commits are easier to work with. Committing each package as it was completed, with a message saying what it did, made it straightforward to look back and see when a behaviour changed. It also meant that when the GPA rewrite went wrong I could see exactly what the previous version had done.

14. Future Enhancements

JDBC and JPA persistence. The clearest next step, and the part of the syllabus this project does not currently cover. A JdbcStorage class implementing the same three operations as FileStorage would let the services run unchanged against MySQL or SQLite, with a proper enrollment table replacing the pipe-separated column and real foreign keys handling the cascade deletes that are currently written by hand. After that, mapping Student, Course and GradeRecord as JPA entities would remove the CSV serialisation code entirely.

Attendance tracking. Attendance is the other number every institute tracks, and it fits the existing model naturally as a second record type keyed on the same student and course pair. Exam eligibility rules based on an attendance percentage would follow from it.

Semester history and progress trends. At the moment the system holds one snapshot. Storing marks per semester would allow a CGPA across semesters, and a simple text chart showing whether a student is improving or slipping.

Authentication and roles. The three actors in the use case diagram are conventions rather than enforced permissions. A login with hashed passwords, and a check that faculty can only enter marks for their own courses, would make those roles real.

Better file handling. Escaping delimiters properly, or moving to a format that handles them, would remove the comma limitation described in section 12.2.

A graphical or web interface. Because the business logic is independent of the console, a JavaFX front end or a Spring Boot REST API could be built over the same services without changing them.

PDF report cards and email. Exporting report cards as PDF and emailing them to mentors would make the analytics useful to people who never open the application.

15. References

1. Schildt, H. (2018). *Java: The Complete Reference*, 11th edition. Oracle Press. (Prescribed text; used for the chapters on inheritance, interfaces, exception handling and multithreading.)
2. Deitel, P. and Deitel, H. (2015). *Java How to Program (Early Objects)*, 10th edition. Pearson.
3. Horstmann, C. S. and Cornell, G. (2011). *Core Java Volume I – Fundamentals*, 8th edition. Pearson. (Referred to for the collections framework and the Comparable/Comparator distinction.)
4. Oracle. *Java SE Documentation and API Specification*. <https://docs.oracle.com/en/java/javase/>
5. Oracle. *The Java Tutorials: Concurrency*. <https://docs.oracle.com/javase/tutorial/essential/concurrency/>
6. Oracle. *The Java Tutorials: Basic I/O*. <https://docs.oracle.com/javase/tutorial/essential/io/>
7. VIT. *CSE2006 Programming in Java – Course Syllabus*. Prepared by Dr. Lakshmi D, compiled by Dr. M. Ashwin and Dr. Sandip Mal.
8. VITYarathi. *Build Your Own Project – General Project Instructions and Submission Guidelines*.

Appendix – Repository

The complete source code, README.md, statement.md and the design diagrams are available in the GitHub repository listed on the cover page. The repository is organised as follows:

```
Smart-Student-Management-System/
  README.md      project overview, setup and testing instructions
  statement.md   problem statement, scope, target users, features
  .gitignore
  run.sh / run.bat  compile and run
  test.sh         compile and run the test suite
  docs/diagrams.md all design diagrams in Mermaid source form
  data/          CSV files, created at runtime
  src/com/sms/    17 source files across 7 packages
```

To build and run from a clean clone:

```
git clone https://github.com/<username>/Smart-Student-Management-System.git
cd Smart-Student-Management-System
./run.sh          # or run.bat on Windows
./test.sh         # runs the 38 validation tests
```